

Assignment -2

Name- Krish Agrawal

Roll no.-240559

Part I: Short Answer / Reasoning Questions

Q 1) Ans. The given query always returns 0 because NULL does not represent a value. It represents missing or unknown information. In SQL, comparing anything with NULL using operators such as =, !=, < or > does not evaluate to TRUE; instead, it evaluates to UNKNOWN. Since the WHERE clause only selects rows where the condition is TRUE, no rows satisfy phone_number = NULL.

The correct query is:

```
SELECT COUNT(*)  
FROM Student  
WHERE phone_number IS NULL;
```

NULL represents an unknown or missing value rather than an actual value. Because its value is unknown, SQL cannot determine the result of normal comparisons involving NULL, so such comparisons evaluate to UNKNOWN instead of TRUE or FALSE. Similarly, most aggregate functions such as SUM(), AVG(), MIN(), and MAX() ignore NULL values because including unknown values could produce incorrect or misleading results.

Q 2) Ans. Yes, the student is correct. In this query, the DISTINCT keyword is redundant because the GROUP BY clause already produces one row for each unique department. GROUP BY already guarantees uniqueness of the grouped columns, so adding DISTINCT does not change the output and only makes the query unnecessarily verbose.

Q3) Ans. A difference in the number of rows indicates that some customers do not have matching records in the Order table. The extra rows returned by the LEFT JOIN represent customers who exist in the Customer table but have no corresponding record in the Order table. This difference reveals the presence of unmatched rows in the left table

Q 4) Ans. The query is invalid because employee_name is neither included in the GROUP BY clause nor used in an aggregate function.

If SQL allowed this query, it would not know which employee's name to display for each department because a department can contain multiple employees. There is no single correct value for employee_name within a group.

Correct Query:

```
SELECT department, AVG(salary)  
FROM Employee  
GROUP BY department;
```

Q5) Ans. The error is that AVG(salary) is used in the WHERE clause. The WHERE clause is evaluated before grouping and aggregation, so aggregate functions cannot be used there.

To filter based on aggregate values, the HAVING clause must be used because it is evaluated after GROUP BY.

Correct Query:

```
SELECT department, AVG(salary)  
FROM Employee  
GROUP BY department  
HAVING AVG(salary) > 80000;
```

Q6) Ans. (a) In this we use non-correlated subquery because the overall average price is calculated once and compared with every product.

```
SELECT *  
  
FROM product  
  
WHERE price > (SELECT AVG(price) FROM product);
```

(b) In this we use correlated subquery because the average salary depends on the department of each employee, so the subquery must reference the current row of the outer query.

```
SELECT *  
  
FROM employee AS e  
  
WHERE salary > ( SELECT AVG(salary)  
_____ FROM employee  
_____ WHERE department =e.department );
```

A correlated subquery is generally slower on large tables because it may be executed once for every row in the outer query. A non-correlated subquery is usually more efficient since it is evaluated only once.

Part II: Long Answer Question

(a) Ans.

```
SELECT *  
FROM Customer as c  
LEFT JOIN Order as o  
ON c.customer_id=o.customer_id  
WHERE o.order_id IS NULL;
```

A LEFT JOIN is used so that all customers are included, even those without matching orders. Customers whose order_id is NULL are the ones who have never placed an order.

(b) Ans.

```
SELECT restaurant_id,  
AVG(order_value) AS average_order_value  
FROM (  
SELECT o.order_id,  
o.restaurant_id,  
SUM(m.price * oi.quantity) AS order_value  
FROM Orders o  
JOIN OrderItem oi  
ON o.order_id = oi.order_id  
JOIN MenuItem m  
ON oi.item_id = m.item_id  
GROUP BY o.order_id, o.restaurant_id  
) AS OrderValues  
GROUP BY restaurant_id  
HAVING COUNT(*) > 5;
```

The inner query calculates the value of each order by summing price × quantity. The outer query computes the average order value for each restaurant, and HAVING is used because the condition (COUNT(*) > 5) applies after grouping.

(c) Ans.

SELECT restaurant_id, AVG(order_value) AS average_order_value

FROM (

SELECT o.order_id,

o.restaurant_id,

SUM(m.price * oi.quantity) AS order_value

FROM Orders o

JOIN OrderItem oi

ON o.order_id = oi.order_id

JOIN MenuItem m

ON oi.item_id = m.item_id

GROUP BY o.order_id, o.restaurant_id

) AS OrderValues

GROUP BY restaurant_id

HAVING AVG(order_value) >

(

SELECT AVG(order_value)

FROM (

SELECT o.order_id,

SUM(m.price * oi.quantity) AS order_value

FROM Orders o

JOIN OrderItem oi

ON o.order_id = oi.order_id

JOIN MenuItem m

ON oi.item_id = m.item_id

```

GROUP BY o.order_id
) AS PlatformOrders
);

```

A non-correlated subquery is used because the platform-wide average order value is calculated only once and does not depend on each restaurant. This is more efficient than a correlated subquery.

(d) Ans.

```

SELECT restaurant_id, item_name, total_quantity
FROM (
    SELECT m.restaurant_id,
           m.item_name,
           SUM(oi.quantity) AS total_quantity,
           RANK() OVER (
               PARTITION BY m.restaurant_id
               ORDER BY SUM(oi.quantity) DESC
           ) AS rnk
    FROM MenuItem m
    JOIN OrderItem oi
    ON m.item_id = oi.item_id
    GROUP BY m.restaurant_id, m.item_name
) AS RankedItems
WHERE rnk = 1;

```

The query first calculates the total quantity ordered for each menu item. The RANK() window function ranks items within each restaurant, and only the highest-ranked item is selected.

Part III: Normalization

Q1) Ans. The following functional dependencies are implied:

- student_id → student_name

- $\text{course_id} \rightarrow \text{course_name}, \text{instructor_id}$
- $\text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office}$
- $(\text{student_id}, \text{course_id}) \rightarrow \text{grade}, \text{enrollment_date}$

The candidate key of the table is $(\text{student_id}, \text{course_id})$ because together they uniquely identify each enrollment.

Q2) Ans. Yes, the table is in First Normal Form (1NF).

Each attribute contains a single (atomic) value, there are no repeating groups or multi-valued attributes, and each row represents one unique student-course enrollment.

Q3) Ans. Since the primary key is $(\text{student_id}, \text{course_id})$, the following are partial dependencies:

- $\text{student_id} \rightarrow \text{student_name}$
- $\text{course_id} \rightarrow \text{course_name}, \text{instructor_id}$

These violate 2NF because non-key attributes depend on only part of the composite key instead of the entire key.

Example of anomaly:

If a course name changes (e.g., *DBMS* to *Database Systems*), every enrollment record for that course must be updated. If one row is missed, inconsistent course names will exist in the database (an update anomaly).

Q4) Ans. The transitive dependency is:

$\text{course_id} \rightarrow \text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office}$

Here, instructor_name and instructor_office depend on course_id indirectly through instructor_id , which violates 3NF.

Example of anomaly:

If an instructor moves to a new office, the office number must be updated in every row for every course enrollment of that instructor. Missing an update leads to inconsistent office information (update anomaly).

Q5) Ans. 3NF Relations

- Student(*student_id*, student_name)
- Instructor(*instructor_id*, instructor_name, instructor_office)
- Course(*course_id*, course_name, instructor_id)
- Enrollment(*student_id*, *course_id*, grade, enrollment_date)

Part IV: Design Justification

In **Part II(c)**, the query could have been written using either a JOIN or a subquery. I chose a non-correlated subquery because the platform-wide average order value is calculated only once and then compared with each restaurant's average order value. This approach is easier to understand and avoids unnecessary complexity. A correlated subquery would recalculate the average for every restaurant, making it less efficient, especially for large datasets.

In **Part III**, normalizing the Enrollment table into Student, Course, Instructor, and Enrollment tables removes redundancy and prevents update, insertion, and deletion anomalies. However, it also makes queries more complex because retrieving complete enrollment information now requires joining multiple tables instead of reading from a single table. This can increase query complexity and execution time. In real-world systems, some denormalization may be intentionally used in reporting or analytics databases, where faster read performance is more important than minimizing redundancy, especially when the data changes infrequently.